

Portfolio Quant Engine

A broker-agnostic quant and portfolio-intelligence platform: mean-variance optimisation over a live holdings book, a two-stage screener across the whole NSE universe, a multi-stage LLM research funnel, and the same Python backend running inside an Android app.

CORE

Python 3.11/3.12 · pandas · SciPy

SURFACES

FastAPI web · Typer CLI · Compose app

BROKERS

Upstox · Groww, either/or

STORE

SQLite + FTS5 · optional TimescaleDB

REASONING

NVIDIA NIM → local fallback chain

SIZE

≈ 14,500 lines across src/ and main.py

Research project. Not investment advice. No credentials, keys or personal data live in the repository — everything sensitive is in a git-ignored `.env` or on the device.

01 One engine, three surfaces, no server

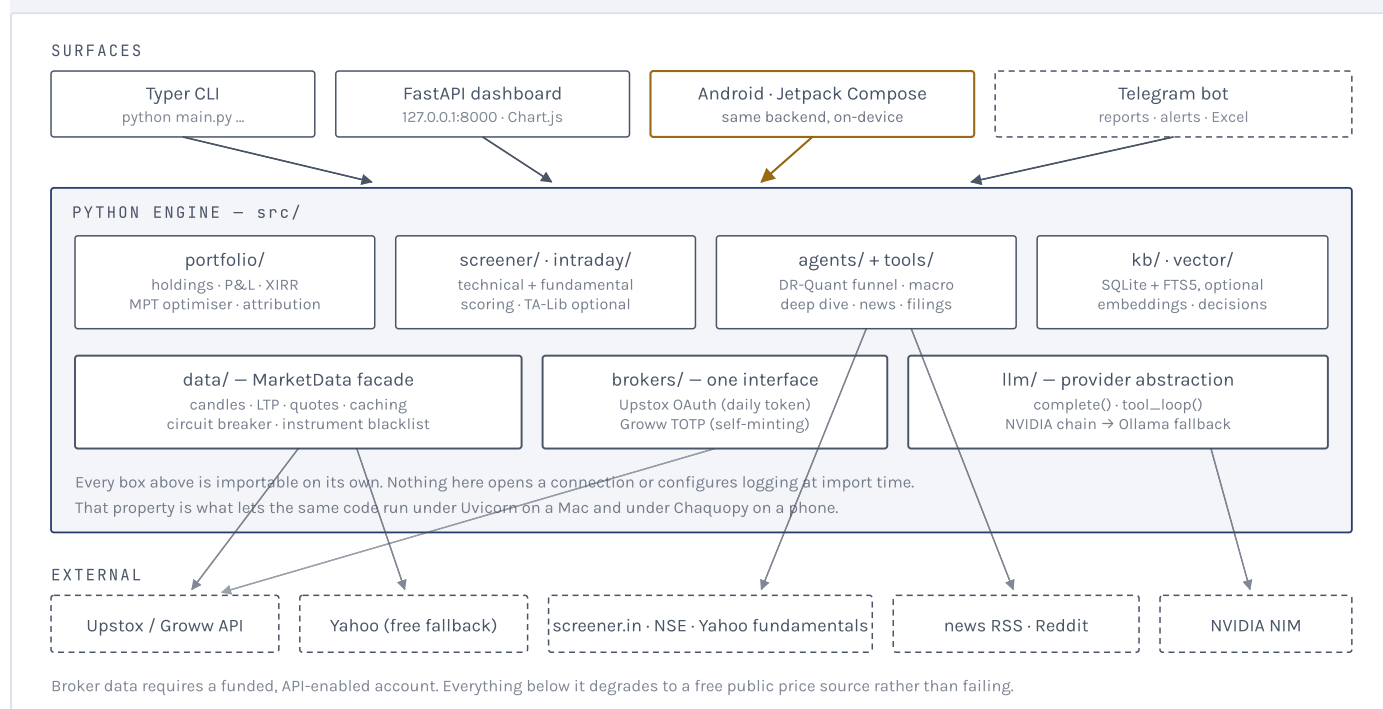
The organising decision is that every module is independently importable and none of them know about the UI. The CLI, the FastAPI dashboard and the Android app are three thin callers over the same functions.

That is easy to claim and hard to hold. It holds here because of a rule applied consistently: **compute modules take DataFrames and dicts in and return dicts out**. No module opens a database connection at import time, none configures logging on the root logger, and none reads the UI's state. The seams where I/O happens — brokers, market data, the knowledge base — are explicit and few.

The payoff is the Android build. A Gradle task copies `src/`, `config/` and the `.env` into the APK, Chaquopy provides a CPython runtime, and the phone runs the identical FastAPI backend against the identical analytics code. There is no mobile fork, no reduced feature set, and no second implementation to keep in sync – which is the failure mode that kills most "and it works on mobile too" projects.

The second organising decision is *graceful degradation everywhere*. A missing broker entitlement, an absent TA-Lib install, an unreachable LLM, a fundamentals page that fails to parse – each has a defined fallback, and the fallback is reported rather than hidden. Section 03 covers the most developed instance of this, the market-data circuit breaker.

02 Architecture



Layering, not a monolith. The two heavy boxes on the second row – `data/` and `brokers/` – are the only places that talk to a broker, so the entitlement problems that dominate Indian retail APIs are handled once rather than at every call site.

03 Brokers, market data, and failing over gracefully

Indian broker APIs share two traits that shape this layer: **tokens expire daily**, and **market-data access is a separate paid entitlement** that a perfectly valid account may not have. Both are routine, not exceptional, so the code treats them as normal states.

Two authentication models behind one interface

Upstox – OAuth 2, daily reissue

A browser flow with automatic code capture at `/callback`. The redirect URI must match the developer-app registration exactly; the dashboard exposes an *auth URL* endpoint and an *exchange code* endpoint so the round trip works identically from the desktop browser and from the phone's WebView.

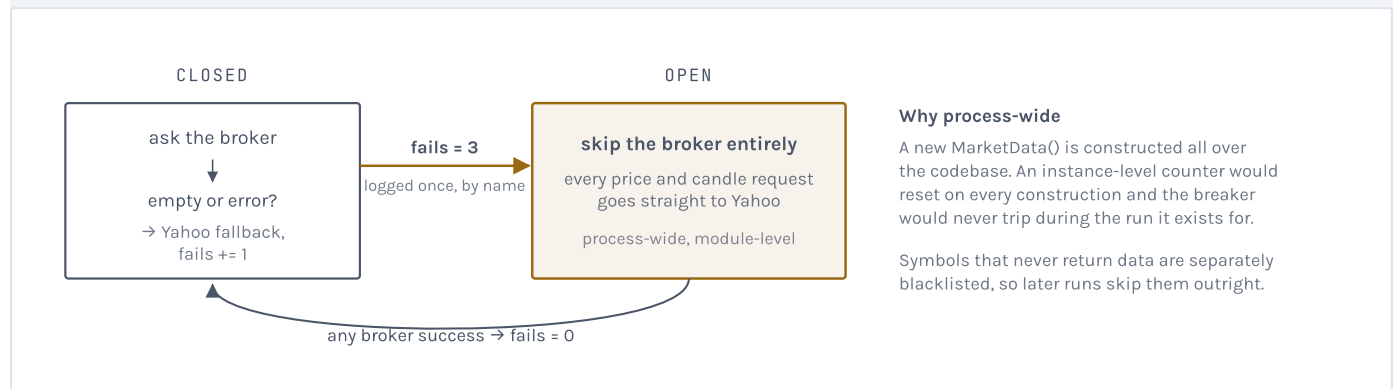
Groww – TOTP, self-minting

Store the base-32 TOTP seed once and the client mints its own daily token, which matters because Groww resets at 06:00 IST. A pasted daily token is also supported for users who would rather not store a seed.

A factory resolves `BROKER=upstox|groww` into an object satisfying one interface, so nothing above this layer branches on which broker is active.

The market-data circuit breaker

A long run – a whole-universe crawl, a DR-Quant funnel – makes thousands of candle requests. If the broker cannot serve historical data (Groww without a data subscription returns 403), the naive behaviour is to try the broker, fail, fall back to Yahoo, and repeat that *for every single symbol*. That is thousands of pointless requests to a service that has already told you no.



Three strikes, then stop asking. The breaker converts a per-symbol failure into a per-run decision. Crucially it is opened by *consecutive* failures and closed by any success, so a couple of genuinely illiquid symbols do not trip it while a missing entitlement does.

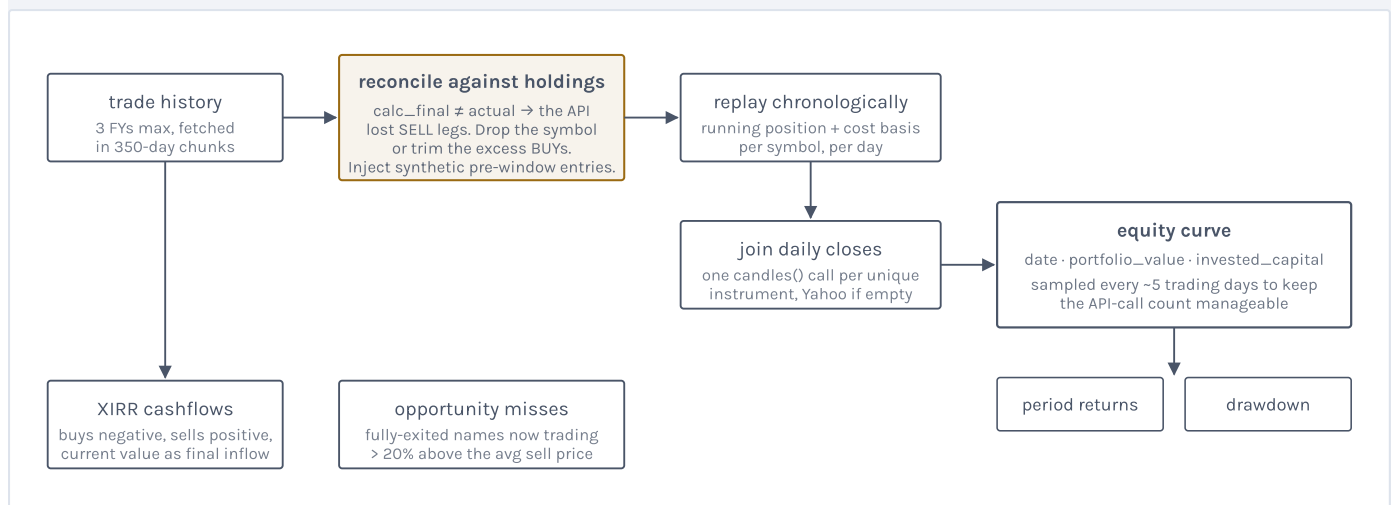
Two smaller mechanisms sit alongside it. Historical requests are cached through a `get_or_set` helper backed by pickled files under `.cache/`, keyed by symbol and window. And an **instrument blacklist** records tickers that return nothing at all, so a 3,000-symbol crawl does not re-attempt the same dead names on every pass.

Concurrency is deliberately low – two worker threads for the screener. Upstox rate-limits hard on a bulk scan; anything that still returns 429 is skipped and picked up on the next incremental run rather than blocking the current one.

Performance: reconstructing what actually happened

A broker tells you what you hold today. It does not tell you what your portfolio was worth last March, what your money-weighted return has been, or which position you sold too early. All three are reconstructed here from the executed-order history.

The equity curve



The repair step is not optional. Reconstruction is arithmetic; the hard part is that the input is wrong in a specific, recoverable way. Everything downstream – returns, XIRR, attribution – inherits whatever that step gets right.

The MTF reconciliation problem

BAD INPUT, QUIETLY

For margin-funded (MTF) positions the Upstox trade history sometimes returns the BUY legs and omits the corresponding SELLS. Replayed naively, the ledger says you still hold a stock you exited months ago – so the equity curve marks a phantom position to market forever and every return figure derived from it is wrong.

The reconciler compares each symbol's *calculated* final quantity against the *actual* holding. A negative gap means missing sells. If the real holding is zero, the entire symbol is dropped as unrecoverable; if it is partially wrong, excess BUY quantity is trimmed from the most recent trades backwards, with the trade's **amount** scaled proportionally so cost basis stays consistent. Positions opened before the API's three-financial-year window get a synthetic entry at estimated historical cost, so the curve does not begin with a cliff.

XIRR

Time-weighted return measures the manager; money-weighted return measures the investor. For a personal book where deposits are irregular, the second is the honest number. XIRR is the rate that zeroes the net present value of the actual, irregularly spaced cashflows:

```
NPV(r) =  $\sum_i A_i / (1 + r)^{(d_i - d_0)/365.25} = 0$   
 $A_i < 0$  for purchases,  $> 0$  for sales; the current portfolio value is  
appended as a final inflow at today's date.
```

Solved with Brent's method over `[-0.999, 100.0]`, which is robust and bracket-guaranteed *when* the interval contains a sign change. It does not always — a book that has only ever received money and never returned any has no root in that range — so a Newton fallback runs from a 0.10 guess, and the function returns `None` rather than a fabricated number if both fail. A missing XIRR displayed as "not computable" is honest; a Newton iterate that wandered somewhere is not.

Opportunity misses

The most useful output in this module is also the simplest: fully-exited symbols whose current price is more than a threshold above the volume-weighted average sell price. It answers "what did I sell too early", which is the question a holdings screen structurally cannot ask, because the position is gone. Current prices are batched in groups of twenty through the LTP endpoint, since Upstox accepts comma-separated instrument keys.

05

The optimiser: mean-variance, and one variant that is actually useful

The standard machinery

Daily returns over a lookback window are annualised at 252 trading days:

```
 $\mu = \text{mean}(r) \times 252$     annualised expected return vector  
 $\Sigma = \text{cov}(r) \times 252$   annualised covariance matrix  
  
portfolio return =  $w^T \mu$   
portfolio vol    =  $\sqrt{w^T \Sigma w}$   
Sharpe          =  $(w^T \mu - r_f) / \sqrt{w^T \Sigma w}$      $r_f$  default 7%, an Indian-  
market rate
```

Three objectives are solved with SLSQP under long-only ($w \geq 0$), fully-invested ($\sum w = 1$) constraints plus a per-name cap defaulting to 25%: **max Sharpe** (minimise negative Sharpe), **min variance**, and **target return** (minimise variance subject to $w^T \mu = \text{target}$). Sampling the third across a range of targets traces the efficient frontier.

Two robustness fixes worth copying

Feasibility auto-relax

With three assets and a 25% cap the weights *cannot* sum to 1 – the problem is infeasible before the solver starts. The code detects `max_weight < 1/n` and relaxes the cap to just past feasible rather than returning a confusing convergence failure.

Multi-start

SLSQP is notorious for reporting "*Positive directional derivative for linesearch*" – a false negative on a problem that has a perfectly good solution. So each objective is attempted from several starting points: equal weights, a seed biased toward the highest- μ third of the assets, and three Dirichlet random seeds from a fixed-seed generator. The first success wins; if none converges, the best non-converged iterate is returned with a debug log rather than an exception.

The fixed RNG seed matters more than it looks: it makes the optimiser *reproducible*. Two runs on the same data return the same weights, which is the difference between a tool you can reason about and one that argues with itself.

The deploy-cash optimiser

This is the variant that earns its place. Textbook MPT tells you the optimal weights and implies you should trade to them – which for a real book means selling winners, realising tax, and paying spreads to reach a theoretical point. The question an investor actually has is narrower and better posed:

"I have ₹X to invest. Given what I already hold, where should it go?"

```

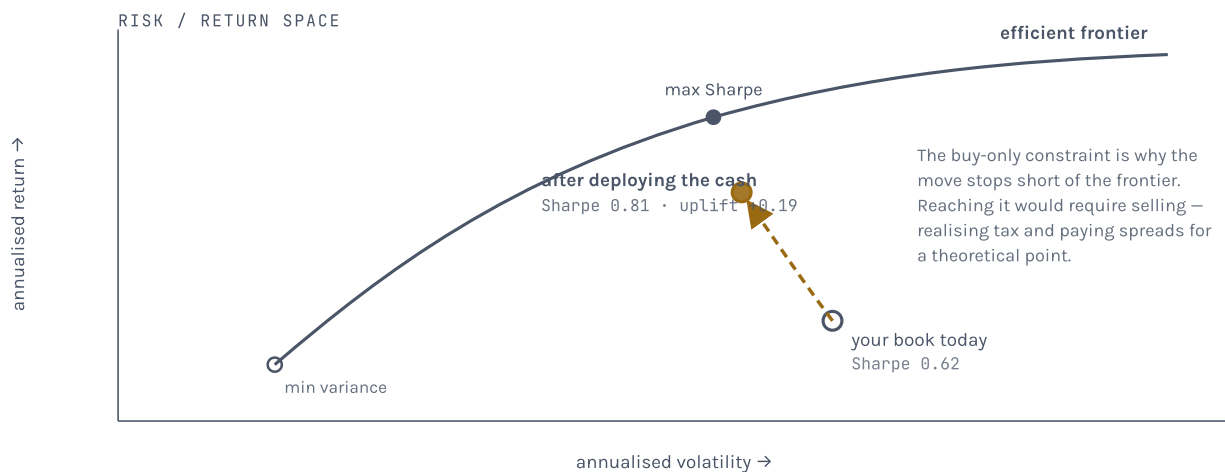
maximise  Sharpe( (v + a) / T )
over      a ∈ ℝn

subject to  ai ≥ 0          buy-only – never sell, so the tax basis is
preserved

           Σ ai = cash      the whole budget is deployed
           (vi + ai)/T ≤ cap  no name exceeds the concentration limit
afterwards

```

where v = current value per holding, $T = \sum v + \text{cash}$, and the per-name upper bound is derived directly as $\max(0, \text{cap} \cdot T - v_i)$ so an already-overweight name simply cannot receive more.



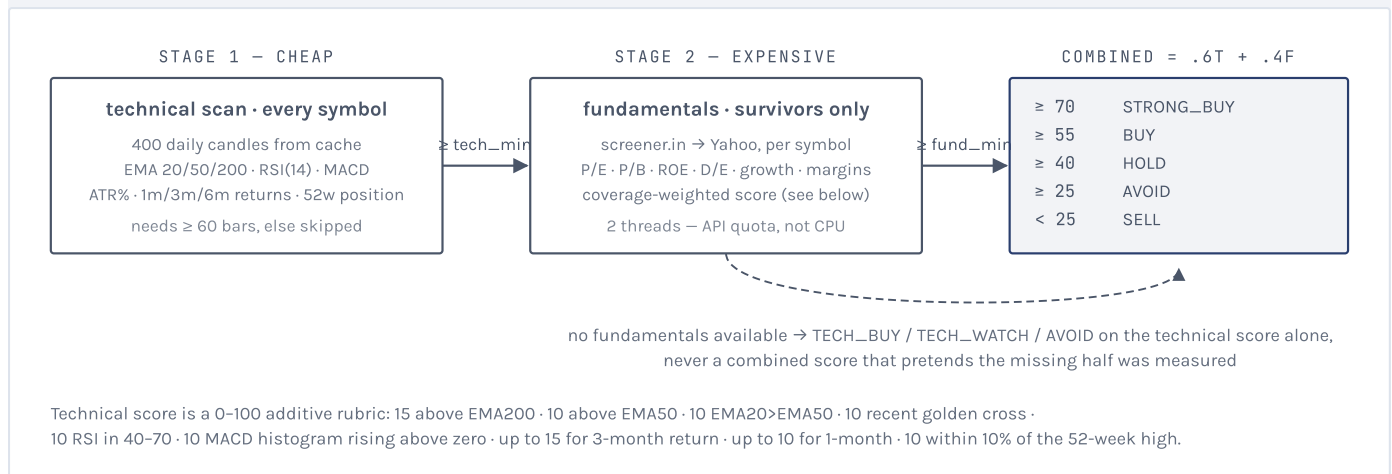
The output is a shopping list, not a target allocation. Every buy is reported in rupees with its resulting weight and whether it opens a new position; the response also carries before/after return, volatility and Sharpe so the uplift is visible rather than asserted. Sub-₹1 allocations are dropped as numerical noise, and the surviving allocations are rescaled so the budget is spent exactly.

`candidates_extra` lets the candidate set include names not yet owned – typically the `STRONG_BUY` rows from the Universe Map (§07), which is how the screener's output becomes an actual portfolio action rather than a list to admire.

The conventional `rebalance_suggestion` also exists – current versus target weight, with a BUY/SELL/HOLD action and a rupee delta per name, sorted by absolute size – for when a real rebalance is intended.

The screener: a funnel shaped by what each stage costs

Screening the whole NSE universe naively means one fundamentals fetch per symbol — roughly 3,000 scrapes of third-party pages, most of them for stocks that fail on price action alone. So the funnel is ordered by cost, not by importance.



Stage order is an economics decision. Stage 1 touches cached candles and runs on everything; stage 2 makes a network request per symbol and runs only on what survived. The dashed path matters as much as the main one: a stock with no obtainable fundamentals is labelled honestly rather than scored as though the missing data were neutral.

Partial-data fundamental scoring

Indian fundamental data is patchy by nature. NSE reliably gives P/E and sector; Yahoo and screener.in add ROE, D/E and margins *when reachable*. A fixed-point rubric caps a P/E-only stock at roughly 15/100 and makes the whole universe look terrible.

So each available signal is normalised to `[0,1]` on its own scale, using one of two shapes:

`band(x, lo, hi)` = 1 inside `[lo, hi]`, linear taper to 0 over one band-width outside

`higher(x, floor, target)` = 0 at floor, 1 at target, linear between

P/E → `band(8, 35)` · P/B → `band(0.5, 8)` · ROE → `higher(5%, 20%)`

D/E → `max(0, 1 - D/E ÷ 2)` · earnings growth → `higher(-5%, 20%)`

revenue growth → `higher(-5%, 15%)` · margins → `higher(0, 20%)` · FCF →

binary

The weighted average is taken over *whatever is present*, then — and this is the part that makes the number usable — scaled by the square root of coverage:

```
raw      =  $\sum(s_i \cdot w_i) / \sum w_i \times 100$     quality across the signals we have
coverage =  $\sum w_i(\text{present}) / \sum w_i(\text{all})$     how much we actually know
score   = raw ×  $\sqrt{\text{coverage}}$ 
```

WHY THE SQUARE ROOT

Without it, every stock that exposed a single reasonable P/E scored 100 and the ranking became meaningless — an artefact that made the screener confidently promote companies about which almost nothing was known. Scaling by $\sqrt{\text{coverage}}$ caps a one-signal score near **35**, two signals near **50**, and only full coverage reaches 100. The square root rather than a linear factor is a deliberate middle: linear punishes partial data so hard that a genuinely good company with two strong signals disappears, while no penalty at all is the bug. The `coverage` value is returned alongside the score, so the caller can always see how much the number rests on.

The setup-based variant

Alongside the additive rubric, `taLib_screener.py` detects named *setups* rather than scoring continuously: RSI bullish divergence (price makes a lower low, RSI a higher low), volume breakout (close above the 20-day high on 1.5× average volume), Wyckoff Phase-C re-accumulation (price wicks below the 30-day low and recovers), and trend-momentum alignment ($\text{EMA}_{20} > \text{EMA}_{50} > \text{EMA}_{200}$ with RSI between 50 and 70). It uses TA-Lib when installed and falls back to pure-pandas implementations otherwise, so a native-library install is an optimisation rather than a dependency. This is what feeds Stage 1 of the research funnel, because a named setup is something an LLM can reason about; a score of 68 is not.

07

The Universe Map: a knowledge base used as a cache

The Universe Map is not a scan you run and read — it is the crawler that fills the knowledge base. Three stages: technical scan over every symbol, multi-source fundamentals for everything technically valid (NSE → Yahoo → screener.in), and a write of each stock into the KB's `universe` store.

It is **incremental by construction**. A stock whose stored entry is younger than `max_age_days` is skipped and its stored data reused. The first build is long; every build after it only refreshes stale names. Run weekly from the scheduler and the KB stays current for free.

The research funnel then reads fundamentals from the KB instead of re-fetching — but with a guard that is easy to get wrong and matters a lot: *a KB entry that is fresh but stores only nulls is worthless*.

The read requires at least one real ratio (P/E, ROE or D/E) present before trusting the cached row; otherwise it falls through to a live fetch and logs which path it took. Cache freshness and cache usefulness are different properties, and only checking the first is how a system ends up confidently reasoning over empty rows.

Output is written twice: a JSON cache under `.cache/universe_map/<universe>.json` for the dashboard's technical-versus-fundamental scatter plot, and the persistent KB store for the funnel and for search.

08

The knowledge base: three stores, zero native dependencies

WHY NOT A VECTOR DATABASE

This started on ChromaDB. Its Rust core caused repeated install and version breakage on an M-series Mac — `'RustBindingsAPI' object has no attribute 'bindings'` — and a research tool that cannot be installed is worth nothing. SQLite's FTS5 full-text index ships *inside* Python's stdlib `sqlite3`: nothing to pip-install, nothing to compile, nothing to version-mismatch. On a phone, where the packaging surface is even narrower, that is not a preference but a requirement.

Three logical stores live in one `.sqlite` file:

`doc_chunks`

Uploaded books, journals, research notes and downloaded filings — chunked and full-text indexed. EPUB and PDF ingestion included.

`decisions`

Every agent verdict, stored with the full prompt, the full response, the verdict, and metadata (run id, macro mode, VIX, technical score, setups, health score). This is a self-accumulating training corpus: `export_decisions()` writes it out for fine-tuning, and `search_decisions()` makes past reasoning retrievable — "why did we reject high-P/E names last quarter" is an answerable question.

`universe`

Per-stock technical and fundamental snapshots with a timestamp, which is what §07 populates and the funnel reads.

Hybrid retrieval, degrading cleanly

Search is a four-step hybrid that works with or without embeddings:

1. Embed the query — via Ollama locally, or NVIDIA NIM's OpenAI-compatible `/embeddings` endpoint when that provider is active. If neither is available, skip to step 4.
2. Build a candidate pool with an FTS5 `MATCH` pre-filter (up to 80 rows). If that returns fewer than 4k rows the keyword filter was too narrow, so widen to every embedded chunk.
3. Re-rank the pool by cosine similarity, stored as float32 blobs and compared with plain NumPy.
4. Drop everything below `kb_min_similarity`. Because results are ranked descending, this is a `break`, not a filter — and if nothing clears the bar, fall through to pure keyword search rather than returning padding.

That last rule is the important one. The LLM should only ever see genuinely relevant context; handing it the five least-bad chunks from an unrelated corpus is how a grounded system starts confabulating with citations.

User queries are sanitised before reaching `MATCH` — FTS5 operators are stripped and surviving alphanumeric terms are OR-ed and quoted — so a query containing a stray `NEAR` or quote character cannot become a syntax error or an injection.

09

The LLM layer

The provider abstraction is two methods. `complete(system, user)` returns text. `tool_loop(system, user, tools, dispatch, max_turns)` orchestrates an agentic loop — the model requests tools, the host dispatches them, the result goes back — and returns a final answer. Four providers implement it: NVIDIA NIM, Ollama, llama.cpp and Anthropic. Claude's native tool use and Ollama's ReAct-style JSON loop are both hidden behind the same signature.

A chain, not a single provider

With `LLM_PROVIDER=nvidia`, the factory builds a fallback ladder rather than one client: each configured NVIDIA model becomes a rung, and a **local model is appended as the final rung**. The cloud model is the main brain; the local one is the safety net that keeps a research run alive through a rate limit or an outage.

On Android the local rung is skipped — detected by the presence of `APP_FILES_DIR`, which the Kotlin layer sets — because there is no Ollama on the phone. That is a one-line environment check standing in for what would otherwise be a platform branch threaded through the whole layer.

The shared KB tool

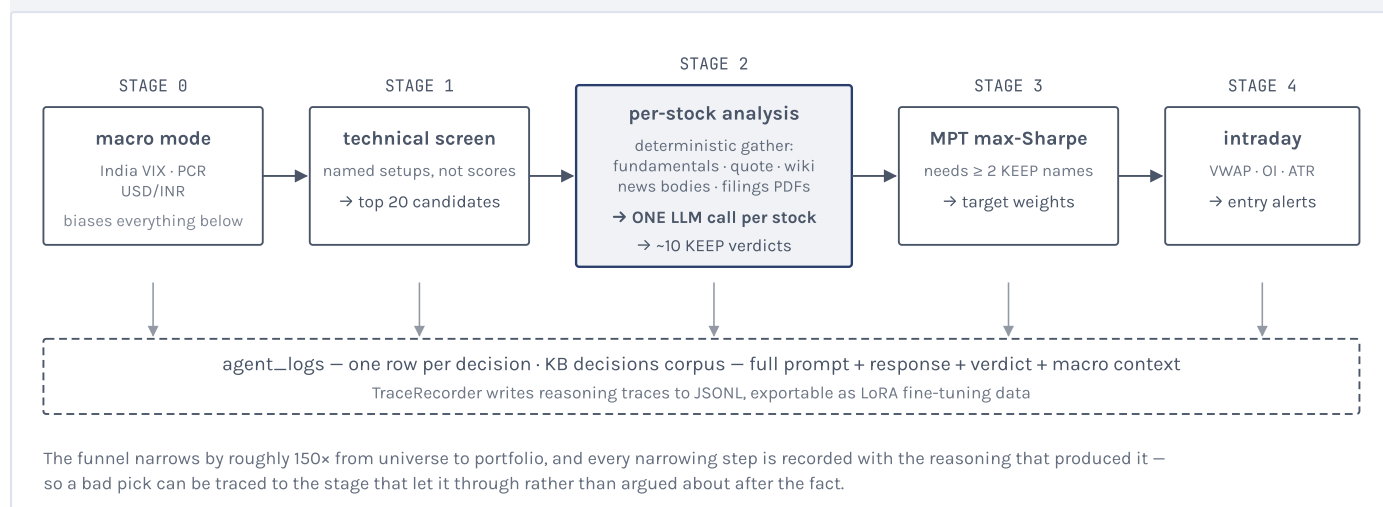
Every agent inherits `kb_search` from `BaseAgent`, but with a conditional that is worth noting: **the tool is only exposed if the KB actually has indexed chunks**. An empty knowledge base means the tool is not advertised at all, saving the model a wasted turn discovering there is nothing there. When it is

exposed, the system prompt is augmented with the chunk count and an instruction to cite the result's title – so a retrieved claim arrives with a source attached.

10

DR-Quant: a five-stage research funnel

The funnel is a small state machine – each stage is a node reading and writing a shared `QuantState` dataclass, and every decision emits an `agent_logs` row (TimescaleDB, or JSONL when no database is configured). The docstring is explicit that this is the LangGraph model without the dependency: a stateful graph of nodes, hand-rolled because the abstraction is smaller than the library.



Recording the reasoning is a design goal, not instrumentation. Because every verdict is stored with its full prompt and the macro conditions at the time, the system accumulates its own evaluation set – which is the prerequisite for ever fine-tuning a model on it.

The shift that matters: agentic → deterministic gather

Stage 2 originally ran as a genuine tool-using agent – an `IntelligenceAgent` with `fundamentals`, `news` and `market_snapshot` tools, reasoning about which to call. That class still exists and its system prompt is a good example of the pattern: explicit REJECT rules (D/E > 1.5 with banks excepted, ROE < 8% and negative sales growth, news mentioning fraud or regulatory action, multi-year low with no catalyst), an explicit output schema, and a hard turn budget with "do not call the same tool twice".

The production path does something different: **gather everything deterministically in Python, then make exactly one LLM call.** The gather is fixed – KB-cached fundamentals with a live-fetch fallback, an Upstox quote, a Wikipedia company description, multi-source news with article bodies, and up to two primary-source filings downloaded, text-extracted and ingested into the KB – and the model's entire job is judgement over an assembled dossier.

Three reasons, in order of weight. **Cost and latency are bounded:** twenty stocks is twenty calls, not twenty times an unpredictable tool loop. **The gather is auditable:** when a verdict is wrong you can inspect exactly what the model saw, because the same code produced it every time. **Tool loops fail in the least useful way:** a model that decides not to call `fundamentals` produces a confident verdict with no fundamentals in it, and nothing in the output says so. Agentic tool use is the right shape when the next action genuinely depends on the previous result; here it never did — the data needed for an equity verdict is known in advance.

One prompt-engineering detail survived from that debugging: the key metrics are surfaced as an explicit aligned key-value block *above* the JSON dossier. When the numbers were only inside the JSON, the model frequently returned `null` for fields that were plainly present — parsing lazily. Spelled out as `P/E : 24.1` it copies them through. That is a real and repeatable property of instruct models handling nested structures, and it costs twelve lines to work around.

11

Macro ideas and the deep dive

Freshness as a hard filter

The macro engine ingests from three source families — around a dozen financial-news RSS feeds, Google News over twelve curated macro queries that move Indian equities, and six Indian market subreddits — plus the live VIX / PCR / USDINR snapshot. Every item is **date-parsed and dropped if older than the requested window**. That filter is the feature: an LLM handed a mix of yesterday's and last quarter's headlines will weight them equally, and stale information biasing a current call is worse than having less of it.

The LLM's job is then to distil those signals into a small number of investable *themes*, each mapped to sectors and then to specific stocks drawn **only from the user's own Universe Map**. That constraint keeps the output actionable: a theme that resolves to a stock you cannot screen, price or optimise is a magazine article, not a position. Each pick carries a sector, a quant-derived entry price and a multi-factor thesis.

Deep dive: primary sources over commentary

For a single stock, the deep dive pulls the last two quarters' results and earnings-call PDFs, extracts their text, and produces a deliberately *skeptical* equity-research read — financial-health issues, valuation concerns, red flags — alongside a technical entry zone derived from DMAs, support levels, RSI and ATR. Fetched filings are ingested into the knowledge base as they arrive, so the same document is available to every later question without being downloaded again.

The skeptical framing is a prompt choice with a reason behind it. A model asked to "analyse" a company produces balanced, unfalsifiable prose. A model asked to find what is wrong with it

produces a checklist you can verify against the filing — which is the only kind of output worth reading when the alternative is reading the filing yourself.

12 Scheduling and delivery

APScheduler runs the funnel on Indian market hours, all jobs pinned to Asia/Kolkata :

	IST	JOB	OUTPUT
	09:00	Pre-open macro check	Market Mode broadcast to Telegram
	09:30	Full funnel, stages 1-3	The day's candidate portfolio
10:00 · 12:00 · 14:30		Intraday advisor	Fresh entry/exit alerts
	15:15	End-of-day summary	Portfolio Excel via Telegram

Every job is wrapped so a single failure logs a traceback and leaves the scheduler running. A scheduler that dies on one bad job is worse than no scheduler, because you stop noticing it is gone.

The Telegram surface is authorised per chat id and can push reports, alerts and generated Excel workbooks. It is a delivery channel over the same engine — not a separate feature set — which is why an EOD report and a CLI portfolio report produce the same numbers.

13 Running the backend on the phone

The Android app is not a client for a remote server. Chaquopy embeds a CPython runtime in the APK, a Gradle task (syncPythonSource) copies src/ , config/ and the project .env in at build time, and android_runner.py starts the same FastAPI application on-device. The Compose UI then talks to 127.0.0.1 over OkHttp.

What makes this practical rather than a stunt is what the engine *does not* require. No native vector database (§08), no TA-Lib (§06 falls back to pandas), no local model runtime (§09 skips the local rung on-device), and no server process to reach. The dependency choices that look conservative on a desktop are precisely what allow the phone build to exist.

The runner sets environment variables *before* any import, loads the bundled .env without overriding what Kotlin already injected, and warns explicitly when an API key is missing rather than failing later inside an LLM call. The app also ships an in-app system terminal, which is the right affordance for a mobile build: when the backend is a real process rather than a remote service, the user needs to be able to see it start.

Where the machine learning is — and where it deliberately is not

There is no trained model in this repository. No `.pkl`, no fitted weights, no training loop. That is a decision, not a gap, and it is worth stating plainly because "quant platform" invites the opposite assumption.

What is used instead

TECHNIQUE	WHERE	WHY IT IS THE RIGHT TOOL HERE
Constrained convex-style optimisation SLSQP over μ and Σ	MPT weights, efficient frontier, deploy-cash allocation	The objective is specified exactly, so there is nothing to learn. Sharpe maximisation under linear constraints is a solved problem; the engineering effort belongs in robustness (multi-start, feasibility relaxation) rather than in modelling.
Weighted rule scoring normalised signals, coverage-scaled	Technical score, fundamental score, combined recommendation	No labels exist. Nobody has annotated 3,000 NSE stocks with "was this a good buy in hindsight", and generating that label from forward returns bakes in one regime. A rubric is also auditable: a user can disagree with the 15 points for trading above the 200-day EMA, which they cannot do with a coefficient.
Zero-shot embeddings + FTSS nomic-embed-text or NIM, cosine	Knowledge-base retrieval, decision search, news store	Pretrained encoders used off the shelf. A personal finance library is hundreds of documents, not a corpus you fine-tune a retriever on — and the hybrid keyword pre-filter means retrieval still works when no embedding backend is reachable at all.
Instruction-following LLMs NIM primary, local fallback	Stock verdicts, macro themes, deep dives, the grounded chat	Genuinely fuzzy work: reading a filing's prose, deciding what counts as a red flag, weighing a dozen macro signals against each other. All of it is constrained to a JSON schema and grounded in retrieved data, so the model exercises judgement over facts it was handed rather than recalling facts of its own.
The four techniques doing the work that a supervised model might be assumed to be doing.		

The case against a fitted model, stated honestly

- **Non-stationarity.** A classifier fitted on the last two years of Indian equity data learns the last two years' regime. Out-of-sample it is not merely less accurate — it is confidently wrong in the specific way the regime shifted.
- **Label construction is the entire problem.** "Good buy" has to be defined as some forward return over some horizon under some drawdown tolerance, and every one of those choices *is* the strategy. The rubric at least states its choices in the open.
- **Explainability is not optional here.** This tool advises a real person about real money. "Score 68 because it trades above its 200-day EMA, a golden cross formed in the last twenty sessions, and ROE is 19%" is something a user can act on or reject. "Score 68" from a model with a validation AUC of 0.61 is not.

- **Sample size.** A personal book has dozens of positions and hundreds of trades. That is enough to compute performance attribution honestly and nowhere near enough to fit anything without overfitting it.

What is being built toward one

The decisions corpus in §08 is the interesting exception. Every verdict is stored with its full prompt, full response, the macro conditions, and the technical setup that produced the candidate – and `TraceRecorder` keeps the reasoning traces as JSONL exportable in LoRA fine-tuning format. That is the deliberate order of operations: *accumulate a real, timestamped, decision-linked dataset first; consider fitting something to it second*. It is the opposite of the usual sequence, and it is the only version that produces a training set worth having.

Generated from the Portfolio source tree – `src/portfolio/`, `src/screener/`, `src/agents/`, `src/kb/`, `src/brokers/`, `src/data/`, `src/dashboard/`, `android/`. A research and educational project; not investment advice. Market data and LLM calls require internet, and broker data requires a funded, API-enabled account.